1.Write a program to find second max number in an array using streams

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

public class SecondMax {

 public static void main(String[] args) {
  //List<Integer> list = Arrays.asList(5, 9, 11, 2, 8, 21, 1);
  int a[] = {5, 9, 11, 2, 8, 21, 1};
  int secondMax = Arrays.stream(a)
                .boxed()//boxed() method converts primitive streams (like IntStream) into object streams like
Stream<Integer>
                .distinct()
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst()
                .get(); //   Correct for Optional<Integer>
    System.out.print(secondMax);

 }

}
```

.....
.....

2.Second max element in list using streams.

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class SecondMaxForArrayList {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(5, 9, 11, 2, 8, 21, 1);

        int secondMax = list.stream()
                        .distinct()
                        .sorted(Comparator.reverseOrder())
                        .skip(1)
                        .findFirst()
                        .get(); // Optional<Integer> → get()

        System.out.print(secondMax);
    }
}
```

.....
.....

3.Print duplicate elements in given array.

```
import java.util.Arrays;
import java.util.stream.Collectors;

public class PrintDuplicateElementsUsingStreamsInArray {

    public static void main(String[] args) {
        int[] input = {1, 2, 2, 3, 4, 4, 5, 2};

        Arrays.stream(input)
              .boxed()
              .collect(Collectors.groupingBy(i -> i, Collectors.counting()))
              .entrySet().stream()
              .filter(e -> e.getValue() > 1)
              .forEach(e -> System.out.println("Element: " + e.getKey() + " → Count: " + e.getValue()));
    }
}
```

.....
.....

4.Remove Duplicate Elemets Using Streams.

```
package streams;
import java.util.Arrays;

public class RemoveDuplicateElemtsUsingStreams {

 public static void main(String[] args) {

//  int[] input = {1, 2, 2, 3, 4, 4, 5};
//
//        int[] unique = Arrays.stream(input)
//                            .distinct()
//                            .toArray();
//
//        System.out.println("Array without duplicates: " + Arrays.toString(unique));

  String[] input = {"apple", "banana", "apple", "orange", "banana"};

        String[] unique = Arrays.stream(input)
                             .distinct()
                             .toArray(String[]::new);

        System.out.println("Array without duplicates: " + Arrays.toString(unique));

 }
}
```

.....
.....

5.Remove duplicates and reverse order using streams.

```
import java.util.*;
import java.util.stream.Collectors;
```

```
public class TestOne {
    public static void main(String[] args) {
        String[] test = {"apple", "banana", "orange", "apple", "guava"};

        List<String> result = Arrays.stream(test)
            .distinct() // removes duplicates
            .sorted(Comparator.reverseOrder()) // sorts in descending (reverse) order
            .collect(Collectors.toList());

        System.out.println(result); // Output: [orange, guava, banana, apple]
    }
}
```

.....
.....

6.Ascending Sort with Duplicates Removed.

```
import java.util.*;
import java.util.stream.Collectors;

public class TestOne {
    public static void main(String[] args) {
        String[] test = {"apple", "banana", "orange", "apple", "guava"};

        List<String> result = Arrays.stream(test)
            .distinct() // removes duplicates
            .sorted() // sorts in descending (reverse) order
            .collect(Collectors.toList());

        System.out.println(result); // Output: [orange, guava, banana, apple]
    }
}
```

.....
.....

7.Find Min Max in array using streams.

```
import java.util.Arrays;
public class FindMaxMin {
    public static void main(String[] args) {
        int[] arr = { 21, 11, 9, 8, 5, 2, 1 };
//
//        // Store sorted elements in a new array
//        int[] sortedArr = Arrays.stream(arr)
//                                .sorted() // ascending order
//                                .toArray(); // collect into new array
//
//        // Print using lambda
//        Arrays.stream(sortedArr)
//              .forEach(x -> System.out.println(x));
//
//        // Find minimum element
```

```
//        int min = Arrays.stream(arr)
//                        .min()
//                        .getAsInt(); // returns int from OptionalInt
//
//        System.out.println("Minimum element: " + min);

        // Find maximum element
        int max = Arrays.stream(arr)
                        .max()
                        .getAsInt(); // returns int from OptionalInt

        System.out.println("Minimum element: " + max);

    }
}
```

.....
.....

8.Occurance of words.

```
package streams;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class OccuranceWords {

 public static void main(String[] args) {
  List<String> items = Arrays.asList("apple", "banana", "apple", "cherry", "banana");

  Map<String, Long> frequencyMap = items.stream()
      .collect(Collectors.groupingBy(
          Function.identity(),
          Collectors.counting()
      ));

  System.out.println(frequencyMap);
  // Output: {apple=2, banana=2, cherry=1}

 }

}
```

** other way

```
package streams;
import java.util.Arrays;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class OccuranceWordsListOfStrings {
```

```
    public static void main(String[] args) {
        String[][] input = {
            {"Java is good"},
            {"Java is bad"},
            {"Java is extreamse"}
        };

        Map<String, Long> wordCount = Arrays.stream(input)
            .flatMap(Arrays::stream) // Flatten to Stream<String>
            .flatMap(sentence -> Arrays.stream(sentence.split(" "))) // Split each sentence
            .collect(Collectors.groupingBy(
                Function.identity(),
                Collectors.counting()
            ));

        System.out.println(wordCount);
    }
}
```

.....
.....

9.Print Longest Length Word using streams.

```
package streams;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

public class LongestLengthWord {

 public static void main(String[] args) {
  List<String> langs = Arrays.asList("Java", "Python", "C++");

        // Find the word with the largest length
        Optional<String> longest = langs.stream()
                                    .max(Comparator.comparingInt(String::length));

        System.out.println(longest.get());

 }

}
```

.....
.....

10.Find top 3 employee sal from specific city using streams?

```
package streams;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
```

```
public class EmployeeStream {

    public static void main(String[] args) {
        record Employee(String name, String city, Integer sal) {}

        List<Employee> employees = Arrays.asList(
            new Employee("Parush", "Hyd", 31),
            new Employee("Kalpana", "sdpt", 28),
            new Employee("Swamy", "gajwel", 31),
            new Employee("Swamy", "gajwel", 31),
            new Employee("Ilu", "gajwel", 40),
            new Employee("Yakki", "Hyd", 10)
        );

        List<Employee> top3Mumbai = employees.stream()
            .filter(e -> e.city().equals("gajwel"))
            .sorted(Comparator.comparing(Employee::sal).reversed())
            .limit(3)
            .toList();

        System.out.println("Top 3 highest-paid employees from Mumbai:");
        top3Mumbai.forEach(System.out::println);
    }
}
```

.....
.....

11.Stream Operators single employees

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

public class GroupingTwo {

 public static void main(String[] args) {
  record Employee(String name, String city,Integer age) {}

  List<Employee> employees = Arrays.asList(
      new Employee("Parush", "Hyd", 31),
      new Employee("Kalpana", "sdpt", 28),
      new Employee("Swamy", "gajwel", 31),
      new Employee("Ilu", "gajwel", 40),
      new Employee("Yakki", "Hyd", 10)
  );
  //System.out.println(employees);

  //1. group by age

// java.util.Map<Integer, List<Employee>> employesByAgeMap = employees.stream()
//  .collect(Collectors.groupingBy(Employee:: age));
```

```
//
// System.out.println(employesByAgeMap);
//
  //2 . display cities name based on age

//  java.util.Map<Integer, List<String>> citiesByAge = employees.stream()
//             .collect(Collectors.groupingBy(
//               Employee:: age,
//                 Collectors.mapping(Employee:: city, Collectors.toList())
//             ));
//
//         System.out.println(citiesByAge);

  //3.oldest age

//   Optional<Employee> oldest = employees.stream()
//              .max(Comparator.comparingInt(Employee :: age));
//
//   System.out.println(oldest);
//
  //3.youngest age

//   Optional<Employee> youngest = employees.stream()
//              .min(Comparator.comparingInt(Employee :: age));
//
//   System.out.println(youngest);
//
  //4.average of age
//  double averageAge = employees.stream()
//             .collect(Collectors.averagingInt(Employee::age));
//
//         System.out.println("Average Age: " + averageAge);

  //4. 2nd oldest element

//   Optional<Employee> secondOldest = employees.stream()
//              .sorted(Comparator.comparingInt(Employee::age).reversed())
//              .skip(1)
//              .findFirst();
//
//   System.out.println(secondOldest);

  //5. Normal sorting
//
//  List<Employee> sortedByAge = employees.stream()
//             .sorted(Comparator.comparingInt(Employee::age))
//             .collect(Collectors.toList());
//   System.out.println(sortedByAge);

  //6. 1st element based on age

//   Optional<Employee> oldest = employees.stream()
//              .sorted(Comparator.comparingInt(Employee::age).reversed())
//              .findFirst();
//
```

```
//   System.out.println(oldest);
 }
}
```

.....
.....

12.Two objects streams.

```
import java.util.List;
import java.util.stream.Collectors;

public class TwoObjects {

 public static void main(String[] args) {
   record Department(int id, String name) {}
   record Employee(int id, String name, int departmentId) {}
  List<Department> departments = List.of(
      new Department(1, "Engineering"),
      new Department(2, "HR"),
      new Department(3, "Finance")
   );

   List<Employee> employees = List.of(
      new Employee(101, "Alice", 1),
      new Employee(102, "Bob", 1),
      new Employee(103, "Charlie", 2),
      new Employee(104, "David", 3)
   );

   //1.Print all employee names using streams.

//   employees.stream()
//      .map(Employee::name)
//      .forEach(System.out::println);

   //2.Filter employees who belong to the "Engineering" department.
   // Step 1: Find the department ID for "Engineering"

//   int engineeringDeptId = departments.stream()
//       .filter(dept -> dept.name().equals("Engineering"))
//       .map(Department::id)
//       .findFirst()
//       .orElse(-1); // fallback if not found
//
//   // Step 2: Filter employees with that department ID
//   List<Employee> engineeringEmployees = employees.stream()
//       .filter(emp -> emp.departmentId() == engineeringDeptId)
//       .toList();
//
//   // Print results
//   engineeringEmployees.forEach(System.out::println);

   //3. Group employees by departmentId.
```

```
//   java.util.Map<Integer, List<Employee>> grouped = employees.stream()
//         .collect(Collectors.groupingBy(Employee::departmentId));
//
//   System.out.print(grouped);

    //4. Count how many employees are in each department.
//   java.util.Map<Integer, Long> countPerDept = employees.stream()
//         .collect(Collectors.groupingBy(Employee::departmentId, Collectors.counting()));
//
//   System.out.print(countPerDept);

 }
}
```

.....

13.Merge two sorted lists into a single sorted list using Java streams:

```
List<Integer> list1 = Arrays.asList(1, 3, 5, 7, 9);
List<Integer> list2 = Arrays.asList(2, 4, 6, 8, 10);
List<Integer> mergedList = Stream.concat(list1.stream(), list2.stream())
                            .sorted()
                            .collect(Collectors.toList());
```

.....

14.Filter

```
package streams;
import java.util.Arrays;
import java.util.List;

public class Filter {

 public static void main(String[] args) {
   List<String> langs = Arrays.asList("Java", "Python", "C++");
        langs.stream()
            .filter(x -> x.startsWith("J"))
            .forEach(x -> System.out.println(x));
 }

}
```

.....

15.Flatmap

```
package streams;
import java.util.Arrays;
import java.util.List;

public class FlatMap {
```

```
 public static void main(String[] args) {
  List<String> sentences = Arrays.asList("Java is fun", "Streams are powerful");
        sentences.stream()
                .flatMap(x -> Arrays.stream(x.split(" ")))
                .forEach(x -> System.out.println(x));

 }

}
```

.....

16.Occurance each character in a String?

```
Map<String, Long> mapObject = Arrays.stream(s.split(""))
                                .map(String::toLowerCase)
   .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
```

.....

17.Merge two employee objects using strems

```
Map<Integer, Employee> mergedMap = Stream.concat(map1.entrySet().stream(),
map2.entrySet().stream())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (emp1, emp2) -> emp1.getSalary() >= emp2.getSalary() ? emp1 : emp2
    ));
```

2. Filter Employees with salary > 55000

```
List<Employee> highEarners = merged.values().stream()
    .filter(emp -> emp.getSalary() > 55000)
    .collect(Collectors.toList());
```

3. Sort Employees by Salary (descending)

```
List<Employee> sortedBySalary = merged.values().stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .collect(Collectors.toList());
```

5. Average Salary

```
double averageSalary = merged.values().stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);
```

Output:

```
56000.0
```

6. Get Highest Paid Employee

```
Optional<Employee> highestPaid = merged.values().stream()
    .max(Comparator.comparingDouble(Employee::getSalary));
```

Output:

```
Employee{id=3, name='Charlie', salary=60000.0}
```

.....

18.Find first non repeating character string java 8

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FirstNonRepeatingChar {
    public static void main(String[] args) {
        String input = "parushram";

        Character firstNonRepeating = input.chars() //- It returns an IntStream — a stream of int values.

            .mapToObj(c -> (char) c) // Convert int to Character
            .collect(Collectors.groupingBy(
                Function.identity(),          // Group by character
                LinkedHashMap::new,           // Preserve insertion order
                Collectors.counting()         // Count occurrences
            ))
            .entrySet()
            .stream()
            .filter(entry -> entry.getValue() == 1) // Keep non-repeating
            .map(Map.Entry::getKey)                 // Extract character
            .findFirst()                        // Get first match
            .orElse(null);                        // Default if none

        System.out.println(firstNonRepeating != null
            ? "First non-repeating character: " + firstNonRepeating
            : "No non-repeating character found.");
    }
}
```

Step 1: input.chars()

Converts the String into an IntStream (stream of int values).

Each int is a Unicode code point for each character.

For "parushram", .chars() gives:

[p, a, r, u, s, h, r, a, m] => [112, 97, 114, 117, 115, 104, 114, 97, 109]

Step 2: .mapToObj(c -> (char) c)

Converts the IntStream into a Stream<Character> by casting int to char.

Now we have a proper Stream<Character>, which we need for grouping and mapping.

.....
.....